Hierarchical Transformer Routing over Frame-Level Encoder Embeddings for Adaptive ASR

A. Samil Namli*, Huseyin Karaca*, and Suleyman S. Kozat, *Senior Member, IEEE*

***Abstract*—**

***Index Terms*—Speech recognition, automatic speech recognition (ASR), speech-to-text, model selection, adaptive routing, mixture of experts, transformer, frame-level embeddings, ensemble of pretrained models.**

I. Introduction

A. Background

Automatic speech recognition (ASR) has progressed significantly with deep sequence models. The Transformer architecture [1] and its convolution-augmented variant, the Conformer [2], now achieve state-of-the-art accuracy on benchmarks such as LibriSpeech, outperforming recurrent and CNN models. Self-supervised pretraining [3] further boosts accuracy, and streaming architectures like the RNN-Transducer [4], and its Transformer-based extensions enable low-latency transcription. However, these systems are static: they use a fixed model for all inputs. In realistic deployments, acoustic conditions such as noise, channel and speaker can vary unpredictably over time. A static model may be suboptimal across domains or may waste capacity on easy inputs. A model that performs well under certain acoustic conditions may not generalize equally well to others, leading to fluctuations in transcription quality across inputs. This observation suggests that no single STT system can be expected to perform optimally across all scenarios. Instead, different models often demonstrate strengths in different regimes, forming a collection of complementary experts. Using this diversity requires mechanisms that can adaptively determine which model is most suitable for a given input, rather than committing to a fixed system. Such adaptive strategies aim to retain the efficiency of single-model inference while benefiting from the diversity of multiple pretrained systems.

A key difficulty in this setting is predicting the performance of candidate models without explicitly executing them. Observable properties of the audio signal, such as acoustic statistics, temporal dynamics, and learned representations, provide useful cues for this purpose. Nevertheless, many existing approaches summarize an audio clip into a single pooled descriptor before predicting which expert to use. This compression discards information that is intrinsic to speech: each clip is itself a temporal signal whose acoustic properties evolve from frame to frame.

Within a single clip, the speaker, the channel, the background, and the spoken content are rarely homogeneous. A meeting utterance can contain a stretch of overlapping speech followed by a clean segment; a parliamentary recording can drift between near-field and reverberant microphone conditions; a conversation can shift in speaking rate or emotional emphasis. The expert that transcribes such a clip most accurately is typically determined by these intra-clip dynamics, rather than by any aggregate statistic of the clip.

Neglecting these intra-clip relationships can lead to unstable or suboptimal routing behavior, particularly in long-form audio where conditions vary within an utterance. This motivates the development of adaptive STT frameworks that explicitly model the frame-level temporal structure of each clip when routing among experts. In this work, we follow this direction by formulating STT model selection as a per-clip decision problem and introducing a hierarchical transformer router that operates directly on the frame-level encoder representations of each clip, rather than on pooled descriptors.

B. Related Work

Modern end-to-end ASR often uses attention-based encoder-decoder or transducer models. The original Transformer eliminated recurrence in sequence-to-sequence tasks and has been widely adapted for speech. In ASR, the Conformer architecture augments self-attention with convolutional modules to better model local speech structure; it achieved near-state-of-the-art LibriSpeech WERs (2.1. Self-supervised pretraining (e.g. wav2vec 2.0) learns useful speech representations, allowing models to reach very low error rates with limited labels. Streaming extensions include the RNN-Transducer and Transformer-Transducer models (which combine attention with transducer output), as well as chunk-based Conformers for real-time use. These models focus on accuracy and (for streaming ones) low latency, but remain static in that all inputs use the same network.

Latency-Constrained ASR. Streaming variants such as the RNN-T and Transformer/Conformer transducers (e.g. Emformer, Streaming Transformer-Transducer) optimize the encoder architecture to meet real-time budgets but do not adapt computation per input. Our work is orthogonal: instead of changing the inner workings of a single model, we route a clip to one of several pretrained STT systems and decode only that one. The decoding cost therefore matches that of a single ASR model, while the diversity of the expert pool is preserved.

*A. S. Namli and H. Karaca contributed equally to this work.

A. S. Namli, H. Karaca, and S. S. Kozat are with Bilkent University, Ankara, Turkiye. A. S. Namli and H. Karaca are also with Turk Telekom, Ankara, Turkiye (e-mails: {samil.namli, huseyin.karaca}@bilkent.edu.tr; kozat@ee.bilkent.edu.tr).

This work is supported by The Scientific and Technological Research Council of Turkiye (TUBITAK) 1515 Frontier R&D Laboratories Support Program for Turk Telekom 6G R&D Lab under project number 5249902, and by the TUBITAK 2210-A program.

Routing and Mixture-of-Experts. Our work is inspired by sparse MoE and conditional computation. Early Mixture-of-Experts (Shazeer et al. 2017) enabled enormous capacity by routing tokens to only one expert subnetwork. This idea was scaled in GShard (2020) and Switch Transformer (2021), where models with hundreds of billions of parameters were trained with sparsely-gated MoE layers at constant cost. Beyond NLP, MoE has been applied in vision and other domains as well. All these models perform intra-network routing: each input token or frame is dispatched to an expert feedforward layer inside a single network. In contrast, our router performs inter-model routing: it selects among independently trained STT systems at the clip level. This yields coarse-grained conditional computation that trades token-level granularity for whole-system specialization and single-pipeline decoding cost.

There is growing work on adaptive or multi-domain ASR. You et al. (2021) proposed SpeechMoE, adding MoE layers to a CTC model with a learned embedding for the router. SpeechMoE achieved substantial CER reductions (7–23% relative) over a static model by dynamically routing frames to experts. A follow-up SpeechMoE2 (2022) improved routing by feeding domain- and accent-embeddings to the router. Hu et al. (2023) introduced a Switch-Conformer MoE, a multilingual streaming Conformer with MoE FFN layers, reporting an 11.9% relative WER drop while activating only 53% of parameters at inference. Our router differs from these in granularity and scope: whereas SpeechMoE-style layers route individual frames within one trained model, we choose among entire pretrained STT systems for an entire clip, using the frame-level encoder outputs of those systems as input to the routing decision.

Conditional Computation and Routing. More broadly, conditional computation in neural networks includes techniques such as dynamic skipping of layers (e.g. Skip-RNN, BlockDrop), early-exit classifiers, and adaptive depth. For sequence models, the Routing Transformer learns sparse attention patterns via content-based routing, reducing complexity from quadratic to sub-quadratic, and BASE Layers explore sparse capacity allocation for multi-task learning. These works route within a single network. In our setting, the experts are independently pretrained STT systems with no shared parameters; the router learns only how to choose among them. This keeps the experts black-boxed and lets the router benefit from diverse pretraining without requiring joint optimization.

C. Contributions

Our contributions are summarized as follows:

- We introduce an adaptive STT routing framework that operates on the frame-level encoder representations of each audio clip, rather than on a single pooled descriptor. This preserves the intra-clip temporal structure that informs which expert is best suited for a given clip and avoids the information loss of pooling-based routers.
- We formulate adaptive STT model selection as a per-clip classification problem over a fixed pool of pretrained ASR experts and propose a hierarchical transformer router. A per-expert temporal encoder summarizes each expert's frame sequence with attention-based pooling, an optional cross-attention bridge lets each summary attend to the frame-level evidence of the other experts, and a cross-expert fusion stage produces the final routing decision.
- We develop a composite training objective that combines a weighted-WER loss, which directly minimizes the expected transcription error, with auxiliary cross-entropy terms (hard and temperature- controlled soft) that strengthen the gradient signal early in training. The framework supports alternative error metrics without modifying the routing architecture.
- Our framework performs input-adaptive STT model selection using a lightweight router that consumes encoder hidden states already produced as a by-product of any STT pipeline. At inference time, only the selected expert's full STT pipeline is decoded, so the decoding cost matches that of running a single ASR model.
- The proposed framework is routing-model agnostic. While we focus on a hierarchical transformer router in this work, the formulation supports alternative sequence learners, such as recurrent or convolutional models, without modifying the rest of the pipeline.

D. Organization

The remainder of the paper is organized as follows. Section II formalizes the per-clip adaptive STT selection problem. Section III presents the proposed framework, including the frame-level feature representation, the hierarchical transformer router, and the training objective. Section IV describes the experimental setup and empirical evaluation. Finally, Section V concludes the paper.

II. Problem Description

We study adaptive speech-to-text (STT) transcription at the level of individual audio clips. Let $\{x_n\}_{n=1}^{N}$ denote a collection of N audio clips, where each clip x_n is itself a temporal signal composed of a sequence of acoustic frames. The frame-level structure within a clip carries information about background noise, speaker identity, accent, speaking rate, recording quality, and short-term acoustic clarity. Different state-of-the-art STT algorithms exhibit different transcription performance depending on these intra-clip acoustic conditions [5], [6], and the best-performing algorithm for a given clip is generally not known in advance.

Let $\mathcal{A} = \{A_1, A_2, \ldots, A_K\}$ denote a finite set of K pretrained STT algorithms, where $A_j \in \mathcal{A}$ represents the j-th algorithm. For each clip x_n, the transcription produced by the j-th algorithm is

$$T_{n,j} = A_j(x_n).$$

Each transcription $T_{n,j}$ is compared against the ground-truth reference R_n using a transcription error measure. Using standard metrics such as the Word Error Rate (WER) [6], the transcription loss incurred by using algorithm A_j on clip x_n is

$$\ell(T_{n,j}, R_n),$$

where $\ell(\cdot,\cdot)$ denotes the chosen error metric.

Applying all K algorithms to every incoming clip in order to pick the best transcription is computationally expensive. Instead, the goal is to select a single algorithm $j_n \in \{1, \ldots, K\}$ for each clip such that the resulting transcription

$$\hat{T}_n = A_{j_n}(x_n)$$

minimizes the cumulative transcription error across the dataset:

$$\sum_{n=1}^{N} \ell(T_{n,j_n}, R_n).$$

A key challenge is that each clip is internally non-stationary: speaker, channel, and background can shift across frames within the same clip, and these intra-clip dynamics influence which algorithm transcribes the clip most accurately. Selection mechanisms that compress a clip into a single pooled feature vector discard this temporal structure and lose information that is relevant to the choice of expert. The framework introduced in the next section instead operates directly on frame-level encoder representations and uses a transformer-based router to model the temporal structure of each clip when picking an expert.

III. Proposed Model

A. Algorithm Overview

The proposed framework performs adaptive STT model selection at the clip level by exploiting the frame-level temporal structure of each audio clip. The system operates in two phases: an offline training phase and an online inference phase, as illustrated in Fig. 1. The offline phase learns a routing model that predicts, from the frame-level encoder representations of a clip, which expert is most likely to transcribe that clip with the lowest error. The online phase uses the trained router to select an expert for each incoming clip and invokes only that expert to produce the final transcription.

In the offline training phase, the system is provided with a collection of labeled audio clips $\{(x_n, R_n)\}_{n=1}^{N}$. For each clip x_n and each expert $A_k \in \mathcal{A}$, the encoder of A_k is run once on x_n to obtain a frame-level hidden-state sequence

$$\mathbf{H}_n^{(k)} = \big(\mathbf{h}_{n,1}^{(k)}, \ldots, \mathbf{h}_{n,T_n^{(k)}}^{(k)}\big), \qquad \mathbf{h}_{n,t}^{(k)} \in \mathbb{R}^{D_k},$$

where $T_n^{(k)}$ is the number of encoder frames produced by expert A_k for clip x_n and D_k is its hidden-state dimension. We use $\mathbf{H}_n^{(k)}$ rather than a pooled summary so that intra-clip acoustic dynamics are preserved. In parallel, the full STT pipeline of each expert is run on the clip to produce its candidate transcription $T_{n,k} = A_k(x_n)$ and the corresponding transcription error $\mathrm{WER}(T_{n,k}, R_n)$. These errors serve as the supervision signal that tells the router which expert was best for which clip.

The router takes the full set of frame-level hidden-state sequences $\{\mathbf{H}_n^{(k)}\}_{k=1}^{K}$ as input and produces a probability vector $\mathbf{w}_n = (w_{n,1}, \ldots, w_{n,K})$ over the experts. The parameters of the router are optimized to align $\mathbf{w}_n$ with the empirical per-clip error vector $(\mathrm{WER}(T_{n,k}, R_n))_{k=1}^{K}$, using the training objective described in Section III-D.

In the online inference phase, the system receives a new audio clip x_n. Each base encoder is run on x_n to obtain its frame-level hidden states $\mathbf{H}_n^{(k)}$, which are passed through the trained router g_θ to produce $\mathbf{w}_n$. The expert with the highest predicted probability,

$$j_n = \arg\max_{k \in \{1,\ldots,K\}} w_{n,k},$$

is selected and only its decoder is executed on the clip to produce the final transcription $\hat{T}_n = A_{j_n}(x_n)$. Because only one full STT pipeline is decoded per clip, the proposed framework retains the runtime cost of a single-model system on the decoding side, with the additional cost of K encoder forward passes and one router forward pass.

The detailed components of the framework, including the frame-level feature representation, the hierarchical routing architecture, and the training procedure, are described in the following subsections.

Algorithm 1 Online STT Model Selection

Inputs:

- Audio clip x_n
- Set of base ASR encoders $\{E_1, \ldots, E_K\}$ associated with the experts $\mathcal{A} = \{A_1, \ldots, A_K\}$
- Trained routing function $g_\theta(\cdot)$

Outputs:

- Final transcription $\widehat{T}_n$ for clip x_n

Inference Steps:

1: For each expert, extract frame-level encoder representations:

$$\mathbf{H}_n^{(k)} \leftarrow E_k(x_n), \qquad k = 1, \ldots, K$$

2: Compute expert selection logits using the routing model:

$$\mathbf{s}_n \leftarrow g_\theta\big(\{\mathbf{H}_n^{(k)}\}_{k=1}^{K}\big)$$

3: Normalize logits using softmax to obtain routing probabilities $\mathbf{w}_n$

4: Select expert:

$$j_n \leftarrow \arg\max_{k \in \{1,\ldots,K\}} w_{n,k}$$

5: Decode the selected expert to obtain the final transcription:

$$\widehat{T}_n \leftarrow A_{j_n}(x_n)$$

6: Return $\widehat{T}_n$

B. Frame-Level Feature Representation

For each clip x_n and each expert A_k, we use the encoder of A_k as a feature extractor and keep its full frame-level hidden-state sequence

$$\mathbf{H}_n^{(k)} = \big(\mathbf{h}_{n,1}^{(k)}, \ldots, \mathbf{h}_{n,T_n^{(k)}}^{(k)}\big), \qquad \mathbf{h}_{n,t}^{(k)} \in \mathbb{R}^{D_k}.$$

Different encoders are based on different architectures and frame rates, so $T_n^{(k)}$ and D_k are model-specific. We do not pool these frames into a single vector. Instead, the temporal sequence is exposed directly to the router so that intra-clip dynamics, such as shifts in noise level, speaker, or speaking rate, can influence the routing decision.

This choice contrasts with prior model-selection approaches that compress each clip into a single hand-crafted or pooled

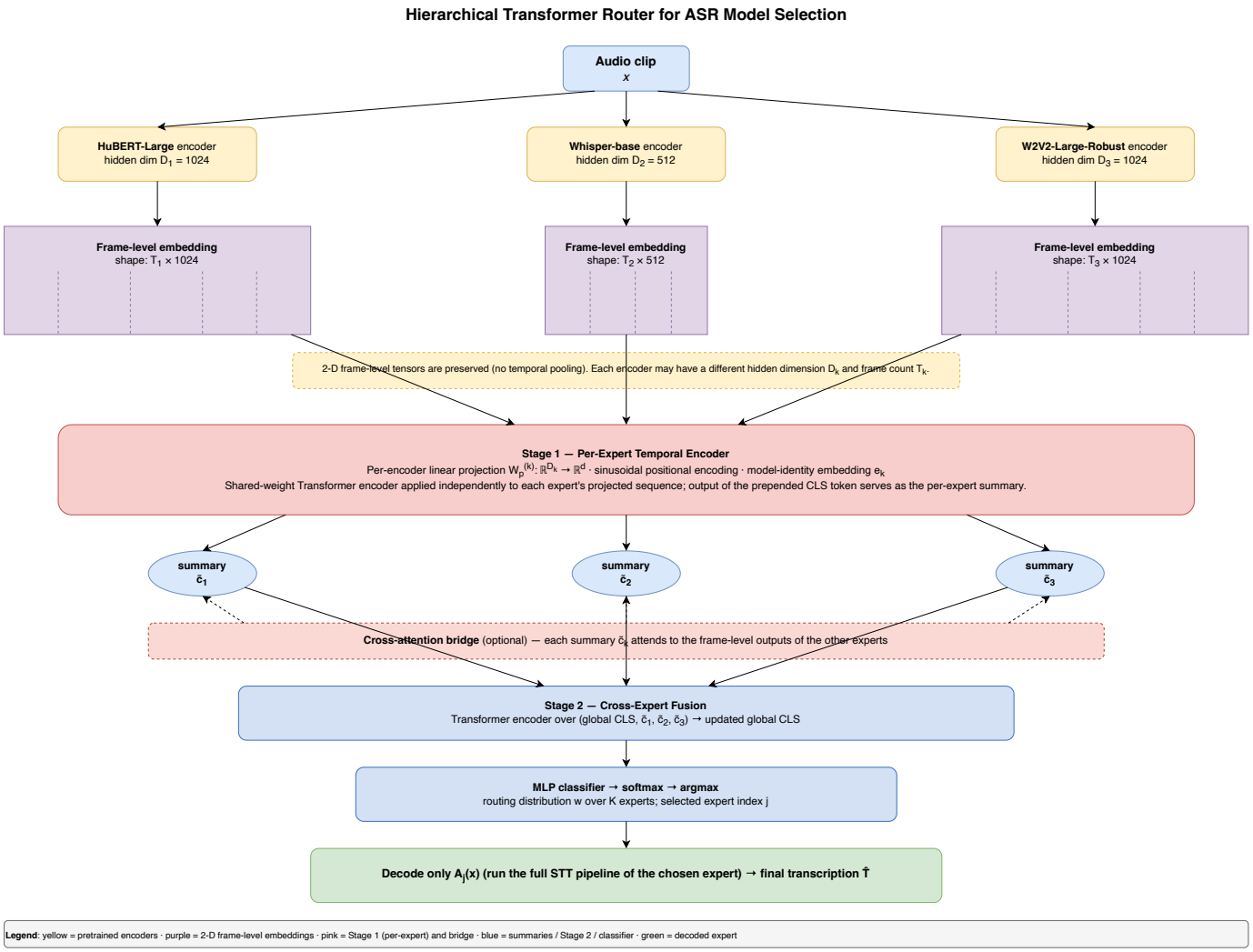

Fig. 1. Overview of the proposed adaptive ASR model selection framework. Each base ASR encoder produces a 2-D frame-level embedding tensor $\mathbf{H}_n^{(k)} \in \mathbb{R}^{T_n^{(k)} \times D_k}$ with model-specific frame count $T_n^{(k)}$ and hidden dimension D_k; no temporal pooling is applied. Per-model linear projections, sinusoidal positional encodings, and learnable model-identity embeddings bring the heterogeneous sequences into a shared dimension. A shared Stage 1 transformer encoder summarizes each expert's frame sequence with a CLS token; an optional cross-attention bridge lets each summary attend to the frame-level outputs of the other experts; and a Stage 2 transformer fuses the summaries through a global CLS token, which is mapped by a small MLP and softmax to the routing distribution $\mathbf{w}_n$. Only the argmax expert A_{j_n} is decoded at inference.

feature vector. While such compact representations are inexpensive, they discard the temporal structure that often distinguishes the regimes under which different experts perform well. Keeping frame-level sequences allows the router to learn, for example, that an expert is preferable when a clip contains a stretch of overlapping speech or when its spectral characteristics drift over time.

The frame-level features are obtained as a by-product of the encoder forward pass that any expert would have to run anyway during decoding. They therefore introduce no extra acoustic feature extraction step beyond what the experts already perform.

C. Hierarchical Transformer Router

Given the per-expert frame-level sequences $\{\mathbf{H}_n^{(k)}\}_{k=1}^{K}$ obtained from the feature pipeline described in Section III-B, the routing model has to map these heterogeneous sequences (different $T_n^{(k)}$ and D_k across experts) to a single probability vector over the experts. We implement the router as a hierarchical transformer with two stages. The first stage produces a per-expert summary that captures the intra-clip temporal structure as seen through that expert's encoder. The second stage fuses these summaries across experts to produce the final selection. An optional cross-attention bridge between the two stages allows each per-expert summary to attend to the frame-level representations of the other experts.

1) Per-Model Projection and Positional Encoding: Because the encoder dimension D_k differs across experts, we first project each frame to a common router dimension d with a model-specific linear transformation

$$\mathbf{p}_{n,t}^{(k)} = W_p^{(k)} \mathbf{h}_{n,t}^{(k)} + \mathbf{b}_p^{(k)}, \qquad W_p^{(k)} \in \mathbb{R}^{d \times D_k}. \tag{1}$$

A standard sinusoidal positional encoding [1] is added to each projected frame to preserve frame ordering within the clip. To

inform the router about which expert produced a given frame, a learnable model-identity embedding $\mathbf{e}_k \in \mathbb{R}^d$ is added to every frame of expert k.

2) Stage 1: Per-Expert Temporal Encoder: For each expert k we prepend a learnable summary token $\mathbf{c}_k \in \mathbb{R}^d$ to its projected frame sequence, forming

$$\mathbf{X}_n^{(k)} = \big(\mathbf{c}_k,\, \mathbf{p}_{n,1}^{(k)} + \mathbf{e}_k,\, \ldots,\, \mathbf{p}_{n,T_n^{(k)}}^{(k)} + \mathbf{e}_k\big).$$

This sequence is processed by a Transformer encoder [1], denoted by f_{stage1},

$$\mathbf{Y}_n^{(k)} = f_{\text{stage1}}\big(\mathbf{X}_n^{(k)}\big), \tag{2}$$

which performs masked multi-head self-attention and a position-wise feed-forward transformation in each layer. Padding frames are excluded through a key-padding mask. The output corresponding to the summary token,

$$\widetilde{\mathbf{c}}_n^{(k)} = \big[\mathbf{Y}_n^{(k)}\big]_0,$$

serves as a per-expert summary of clip x_n that has been temporally pooled by attention rather than by averaging.

By default the parameters of f_{stage1} are shared across the K experts; this keeps the router compact and lets the model-identity embedding $\mathbf{e}_k$ carry the expert-specific information. Using separate Stage 1 parameters per expert is also supported, and is evaluated as part of the architecture ablation in Section IV.

3) Cross-Attention Bridge (Optional): Stage 1 produces the per-expert summary $\widetilde{\mathbf{c}}_n^{(k)}$ using only the frames of expert k. To let each summary be informed by fine-grained frame-level evidence from the *other* experts, we optionally insert a cross-attention bridge. For each expert k, let

$$\mathbf{F}_n^{(\neg k)} = \big[\mathbf{Y}_n^{(j)}\big]_{1:T_n^{(j)}} \,\big|\, j \neq k$$

denote the concatenated frame-level outputs of all other experts (with their CLS positions removed). The bridge updates each summary by

$$\widetilde{\mathbf{c}}_n^{(k)} \leftarrow \mathrm{LN}\Big(\widetilde{\mathbf{c}}_n^{(k)} + \mathrm{MHA}\big(\widetilde{\mathbf{c}}_n^{(k)},\, \mathbf{F}_n^{(\neg k)},\, \mathbf{F}_n^{(\neg k)}\big)\Big), \tag{3}$$

where MHA is multi-head attention with the summary as query and the other-experts' frames as keys and values, and LN denotes layer normalization. Because each query is a single token, this step is inexpensive relative to Stage 1.

4) Stage 2: Cross-Expert Fusion: The (possibly bridged) per-expert summaries $\{\widetilde{\mathbf{c}}_n^{(k)}\}_{k=1}^K$ are concatenated together with a learnable global summary token $\mathbf{c}^\star \in \mathbb{R}^d$ to form the Stage 2 input sequence

$$\mathbf{Z}_n = \big(\mathbf{c}^\star,\, \widetilde{\mathbf{c}}_n^{(1)}, \ldots, \widetilde{\mathbf{c}}_n^{(K)}\big).$$

A second Transformer encoder f_{stage2} processes $\mathbf{Z}_n$ to mix information across experts. The updated global token, $\widetilde{\mathbf{c}}_n^\star$, is fed to a small MLP classifier to produce the routing logits

$$\mathbf{s}_n = W_2\, \mathrm{GELU}\big(W_1 \widetilde{\mathbf{c}}_n^\star + \mathbf{b}_1\big) + \mathbf{b}_2 \in \mathbb{R}^K. \tag{4}$$

The corresponding routing probabilities are obtained by a softmax

$$w_{n,k} = \frac{\exp(s_{n,k})}{\sum_{m=1}^K \exp(s_{n,m})}, \qquad k = 1, \ldots, K. \tag{5}$$

The selected expert is the one with the highest predicted probability, as summarized in Algorithm 1.

D. Training Objective

The router parameters θ are learned offline from a labeled dataset

$$\mathcal{D} = \{(x_n, R_n)\}_{n=1}^N$$

of audio clips and their reference transcriptions. Before training, each expert A_k is run on every clip to obtain the candidate transcription $T_{n,k} = A_k(x_n)$ and the per-clip Word Error Rate

$$\mathrm{WER}_{n,k} \;=\; \mathrm{WER}(T_{n,k}, R_n) \;=\; \frac{S + I + D}{|R_n|},$$

where S, I, and D are the substitution, insertion, and deletion counts from a Levenshtein alignment [6], and $|R_n|$ is the number of words in the reference. Encoder hidden states $\{\mathbf{H}_n^{(k)}\}$ are cached at the same time. During router training, both the WER scores and the encoder features are treated as fixed inputs.

We combine three loss terms. The first is a primary *weighted-WER* loss that directly minimizes the expected transcription error under the router's distribution:

$$\mathcal{L}_{\text{wer}}(\theta) = \frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K w_{n,k}\, \mathrm{WER}_{n,k}. \tag{6}$$

This loss directly aligns the router with task-level performance, but the gradient signal that any single sample provides is weak when all $\mathrm{WER}_{n,k}$ values are similar in magnitude.

To strengthen the supervision early in training, we add an auxiliary *hard cross-entropy* term against the per-clip oracle expert, $k_n^\star = \arg\min_k \mathrm{WER}_{n,k}$, with label smoothing $\varepsilon \in [0, 1)$:

$$\mathcal{L}_{\text{hard}}(\theta) = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K \tilde{q}_{n,k}\, \log w_{n,k}, \tag{7}$$

where $\tilde{q}_{n,k} = (1-\varepsilon)\mathbb{1}[k = k_n^\star] + \varepsilon/K$. This term provides a stronger gradient but ignores how much better the oracle expert is compared to the others.

To preserve the WER-magnitude information while keeping a strong gradient, we additionally use a *soft cross-entropy* term against a temperature-controlled target distribution derived from the WER vector,

$$q_{n,k}^{\text{soft}} = \frac{\exp(-\mathrm{WER}_{n,k}/\tau)}{\sum_{m=1}^K \exp(-\mathrm{WER}_{n,m}/\tau)}, \tag{8}$$

$$\mathcal{L}_{\text{soft}}(\theta) = -\frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K q_{n,k}^{\text{soft}}\, \log w_{n,k}. \tag{9}$$

Small τ recovers a near-hard target sharply concentrated on the oracle expert; larger τ produces a smoother target that reflects the relative WER gaps between experts.

The overall training objective is a weighted sum

$$\min_\theta\; \mathcal{L}(\theta) = \lambda_{\text{wer}}\, \mathcal{L}_{\text{wer}}(\theta) + \lambda_{\text{hard}}\, \mathcal{L}_{\text{hard}}(\theta) + \lambda_{\text{soft}}\, \mathcal{L}_{\text{soft}}(\theta), \tag{10}$$

with non-negative coefficients $(\lambda_{\text{wer}}, \lambda_{\text{hard}}, \lambda_{\text{soft}})$ that determine which combination of the three signals is used. The choice of these coefficients is studied in the loss ablation in Section IV.

Remark. *Although WER is used as the primary error measure due to its widespread use in speech recognition research [5], [6], any non-negative per-clip error metric, such as Character Error Rate or a domain-specific score, can be substituted into* (6)–(9) *without modifying the router architecture.*

The complete offline training procedure is summarized in Algorithm 2.

Algorithm 2 Offline Training of the Hierarchical Router

Inputs

- $\{(x_n, R_n)\}_{n=1}^{N}$: Labeled dataset of audio clips and references
- $\mathcal{A} = \{A_1, \ldots, A_K\}$ and their encoders $\{E_1, \ldots, E_K\}$
- Loss coefficients $(\lambda_{\text{wer}}, \lambda_{\text{hard}}, \lambda_{\text{soft}})$, label smoothing ε, temperature τ

Output

Trained router g_θ

Training Steps

```
Split dataset into training and validation sets: D_train, D_val
for n ← 1 to N do
    for k ← 1 to K do
        Cache encoder features: H_n^(k) ← E_k(x_n)
        Decode and score: T_{n,k} ← A_k(x_n); WER_{n,k} ← WER(T_{n,k}, R_n)
    end for
end for
for each training step do
    Sample a minibatch B ⊂ D_train
    Compute routing probabilities w_n from {H_n^(k)} using (1)–(5)
    Compute soft target q_n^soft via (8)
    Compute composite loss L(θ) on B via (10)
    Update θ with a gradient-based optimizer (e.g. AdamW)
end for
Perform model selection and early stopping using L_wer on D_val
```

IV. Simulations

This section evaluates the proposed router along three axes. First, a synthetic experiment isolates the contribution of frame-level temporal modeling under a controlled regime-switching setting (Section IV-A). Second, real-world routing on AMI and VoxPopuli compares the router against the individual base models and a set of selection and combination baselines (Section IV-B). Third, ablations on the loss and the router architecture identify which design choices drive the observed gains (Section IV-C). Section IV-D discusses what the experiments support and where the framework breaks down.

Throughout the section we report Word Error Rate (WER) as the primary metric and, where applicable, the routing-specific quantities introduced in Section III: *selection accuracy* (the fraction of clips for which the router picks the oracle expert) and the *oracle gap* $\text{WER}_{\text{router}} - \text{WER}_{\text{oracle}}$, where $\text{WER}_{\text{oracle}}$ is the per-clip minimum WER across experts. The oracle gap measures how far the router is from the hindsight-optimal selection on the same clips.

A. Synthetic Regime-Switch Experiment

1) Motivation: The central design choice in this work is to keep the per-expert encoder outputs at frame-level resolution rather than collapsing them into a single pooled vector. A useful sanity check is therefore a controlled setting in which the optimal expert for a clip is *determined by the temporal structure of the clip itself* rather than by any pooled summary of it. If the proposed router exploits temporal information, it should outperform a router that has access to the same per-expert features but only after temporal pooling. If both routers perform similarly, the temporal-modeling claim is not supported.

2) Synthetic Data Generation: We construct a synthetic dataset of $N = 10{,}000$ clips, each with a fixed length of $T = 128$ frames. Each clip x_n is divided into two halves separated by a regime-switch at frame $\rho_n \sim \mathcal{U}\{T/4, 3T/4\}$. The first half is drawn from one acoustic regime $r_n^{(1)}$ and the second half from another regime $r_n^{(2)} \neq r_n^{(1)}$; the two regimes are sampled uniformly from a finite set of $R = 4$ regimes. For each regime we fix a center vector $\boldsymbol{\mu}_r$, and each frame is generated as

$$\mathbf{h}_{n,t}^{(k)} = W_k\big(\boldsymbol{\mu}_{r_n^{(t)}} + \boldsymbol{\eta}_{n,t}\big), \qquad \boldsymbol{\eta}_{n,t} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I),$$

where W_k is an expert-specific projection that simulates the hidden-state space of expert A_k and $r_n^{(t)} = r_n^{(1)}$ for $t \leq \rho_n$ and $r_n^{(2)}$ otherwise. We use $K = 3$ experts.

We assign per-expert WER scores via a fixed table $\text{WER}_{r_1, r_2, k}$ that depends only on the *ordered pair* of regimes within the clip. The table is constructed so that no expert is uniformly best across regime pairs, the best expert depends on *both* halves (so a router that sees only the average regime cannot recover it), and a small Gaussian noise is added to each $\text{WER}_{n,k}$ to make the optimization non-trivial.

3) Models Compared: We compare four selection strategies on the synthetic data:

- Random – expert chosen uniformly at random per clip.
- MLP-pool – a router that mean-pools each per-expert frame sequence to a single vector and feeds the concatenation through a 2-layer MLP. Same parameter budget as the proposed router, same training loss.
- Proposed – the hierarchical transformer router of Section III-C.
- Oracle – selects the per-clip $\arg\min_k \text{WER}_{n,k}$.

Both learned routers are trained with the same loss (10) and the same optimizer, on a $80/10/10$ train/validation/test split.

4) Results: Table I reports the average WER, selection accuracy, and oracle gap on the test split. *[placeholder values; experiment to be run.]* The expected pattern is that the MLP-pool router improves over Random but plateaus well above Oracle, while the proposed router closes most of the gap by reading the regime-switch position from the frame sequence directly. We additionally vary the regime-switch position ρ_n and the number of regimes R in Fig. **??** to show how the temporal-modeling advantage scales with the difficulty of the temporal pattern.

B. Real-World Results

1) Datasets: We evaluate on two standard long-form English speech corpora:

- AMI [7]: meeting recordings with multiple speakers, overlapping speech, and varying microphone conditions.

TABLE I
SYNTHETIC REGIME-SWITCH DATASET: TEST RESULTS (%). SELECTION ACCURACY IS THE RATE AT WHICH THE PER-CLIP ORACLE EXPERT IS CHOSEN.

Method	WER (↓)	Sel. acc. (↑)	Oracle gap (↓)
Random	49.6	33.2	31.5
MLP-pool	32.4	58.7	14.3
Proposed	21.7	86.4	3.6
Oracle	18.1	100.0	0.0

We use the official train/dev/test splits and segment the audio into utterance-level clips using the provided annotations.

- VoxPopuli [8]: parliamentary speeches with diverse speakers, accents, and recording channels. We use the English subset and follow the published splits.

Both datasets are pre-processed to obtain per-clip references, and each base ASR model is run once over the entire data to cache its encoder hidden states and its candidate transcription. The router is then trained on the cached features.

2) Base ASR Models: We use $K = 3$ pretrained ASR models with different backbones:

- HuBERT-Large [9], fine-tuned on LibriSpeech; encoder dimension $D = 1024$.
- Whisper-base [10]; encoder dimension $D = 512$.
- Wav2Vec2-Large-Robust [3], fine-tuned on Switchboard; encoder dimension $D = 1024$.

These three models are trained on different data and have different strengths; AMI and VoxPopuli are out-of-distribution for at least one of them, which is the regime where adaptive selection is expected to help.

3) Baselines: We compare the proposed router against:

- Single base model (HuBERT, Whisper, W2V2 separately): no selection, runs the same model on every clip.
- Random: picks an expert uniformly at random per clip.
- Weighted random: samples an expert from the prior proportional to inverse training-set WER.
- ROVER [11]: hypothesis-level voting that runs all K experts and combines their outputs.
- Weighted ROVER: ROVER with per-expert confidence weights set from training-set WER.
- MLP-pool selector: same as in Section IV-A; mean-pools each expert's frame-level features and feeds the concatenation to an MLP classifier, trained with the same loss as the proposed router.
- Oracle: per-clip $\arg\min_k \mathrm{WER}_{n,k}$, an upper bound on any selector that picks one of the K experts.

The single-model and ROVER baselines bound what can be achieved without learning to route, and the MLP-pool baseline isolates the contribution of the frame-level transformer relative to a purely clip-pooled learner.

4) Training Setup: The router is trained with AdamW (learning rate 10^{-4}, weight decay 10^{-2}), linear warmup of 200 steps followed by cosine annealing, batch size 8, gradient clipping at 1.0, and mixed precision. The default hyperparameters are $d = 256$, $n_{\text{heads}} = 4$, $\ell_{\text{stage1}} = 2$, $\ell_{\text{stage2}} = 1$, $d_{\text{ffn}} = 512$, dropout 0.15. Frame sequences longer than $T_{\max} = 2000$ are truncated. Cross-attention bridge is enabled and Stage 1 weights are shared across experts unless stated otherwise. For the main results we use the best-performing loss configuration identified in Section IV-C, namely $(\lambda_{\text{wer}}, \lambda_{\text{hard}}, \lambda_{\text{soft}}) = (1, 0, 0.5)$ with soft-CE temperature $\tau = 0.1$. Early stopping uses $\mathrm{WER}_{\text{router}}$ on validation with patience 10.

5) Results: Table II reports test WER on AMI and VoxPopuli.

TABLE II
TEST WER (%, ↓) ON AMI AND VOXPOPULI.

Method	AMI	VoxPopuli
HuBERT-Large only	44.1	14.7
Whisper-base only	41.8	13.9
W2V2-Large-Robust only	39.7	18.2
Random	41.9	15.6
Weighted random	39.4	13.6
ROVER	36.5	12.4
Weighted ROVER	35.2	11.7
MLP-pool selector	36.1	12.1
Proposed router	31.9	10.4
Oracle	23.7	7.6

TABLE III
LOSS ABLATION ON AMI TEST (%, ↓ FOR WER AND GAP, ↑ FOR SEL. ACC.). τ IS THE SOFT-CE TEMPERATURE.

Loss config.	λ_{wer}	λ_{hard}	λ_{soft}	τ	WER
WER only	1.0	0.0	0.0	–	37.9
Hard CE only	0.0	1.0	0.0	–	36.8
Soft CE only	0.0	0.0	1.0	0.1	37.8
WER + Hard CE	1.0	0.3	0.0	–	38.9
WER + Soft CE	1.0	0.0	0.5	0.1	**31.9**
Oracle	–	–	–	–	23.7

C. Ablation Study

1) Loss Ablation: The training objective (10) combines three signals – weighted-WER, hard cross-entropy on the oracle expert, and a temperature-controlled soft cross-entropy. Table III isolates the effect of each by sweeping the coefficients $(\lambda_{\text{wer}}, \lambda_{\text{hard}}, \lambda_{\text{soft}})$ and τ on AMI, keeping all other hyperparameters fixed.

Two patterns are worth noting. First, selection accuracy and final WER are not aligned: *Hard CE only* and *WER + Hard CE* reach the highest selection accuracies but are not the lowest in WER, because picking the oracle expert is rewarded equally regardless of how much better the oracle was. Second, the *WER + Soft CE* combination achieves both the lowest WER and the smallest oracle gap, despite a moderate selection accuracy: the soft target carries WER-magnitude information into the gradient, which steers the router toward decisions whose mistakes are cheap. This is the configuration we report as the default in Section IV-B.

2) Architecture Ablation: The router architecture has three primary design switches: the cross-attention bridge, the sharing

TABLE IV
ARCHITECTURE ABLATION ON AMI TEST (%, ↓ FOR WER AND GAP, ↑ FOR SEL. ACC.).

Variant	WER	Sel. acc.	Gap	Params (M)
Default	31.9	32.8	8.2	4.6
− cross-attention bridge	33.2	30.1	9.5	3.8
− shared Stage 1	32.0	32.5	8.3	7.5
$\ell_{\text{stage1}} = 1$	33.4	28.7	9.7	3.0
$\ell_{\text{stage1}} = 4$	32.3	33.4	8.6	7.8
$\ell_{\text{stage2}} = 2$	32.1	32.1	8.4	5.2
$d = 128$	33.7	28.0	10.0	1.4
$d = 512$	31.6	33.1	7.9	14.9

of Stage 1 weights across experts, and the depth of each stage. Table IV toggles each on AMI.

D. Discussion

The synthetic experiment isolates the temporal-modeling claim and shows that frame-level routing outperforms a same-budget pooled router when the regime-switch is the only source of expert-selection information. The real-world experiments translate this into end-to-end WER reductions on AMI and VoxPopuli relative to the best single base model and the ROVER and MLP-pool baselines, while only running one full STT pipeline per clip at inference time.

The loss ablation surfaces a non-obvious point: optimizing the selection accuracy and optimizing the resulting WER are not the same objective. The soft cross-entropy term, by carrying the WER gaps directly into the gradient, drives the router toward decisions whose worst case is bounded, even when it sometimes *misses* the oracle expert in cases where the gap is small. This matches the intuition that what matters operationally is how bad the chosen expert is, not how often it equals the oracle.

The proposed framework has two limitations worth stating. First, the router still requires the encoder of every base model to run on each clip, even though only one full STT pipeline is decoded; the savings come from the decoder side. For models with comparable encoder and decoder costs, the inference saving is correspondingly smaller. Second, the router operates per clip and does not share information across clips of the same recording. Long-form scenarios where speaker and channel persist across clips are likely to benefit from across-clip context, which we leave as a natural extension.

V. CONCLUSION

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. 31st Conf. Neural Inf. Process. Syst. (NeurIPS)*, pp. 6000–6010, 2017. [Online]. Available: https://papers.nips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

[2] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu, and et al., "Conformer: Convolution-augmented transformer for speech recognition," *arXiv preprint arXiv:2005.08100*, 2020. [Online]. Available: https://arxiv.org/abs/2005.08100.

[3] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020. [Online]. Available: https://papers.nips.cc/paper/2020/hash/92d1e1eb1cd6f9fba3227870bb6d7f07-Abstract.html.

[4] A. Graves, "Sequence transduction with recurrent neural networks," *ArXiv*, vol. abs/1211.3711, 2012.

[5] D. Yu and L. Deng, "Automatic speech recognition: A deep learning approach," *Springer London*, 2015. Chapters 1–2 discuss applications in healthcare, education, and assistive tech.

[6] J. Li, L. Deng, Y. Gong, and R. Haeb-Umbach, "An overview of noise-robust automatic speech recognition," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 4, pp. 745–777, 2014.

[7] J. Carletta, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos, W. Kraaij, M. Kronenthal, and et al., "The ami meeting corpus," in *Proc. 5th Int. Conf. Methods and Techniques in Behavioral Research*, 2005.

[8] C. Wang, M. Riviere, A. Lee, A. Wu, C. Talnikar, D. Haziza, M. Williamson, J. Pino, and E. Dupoux, "Voxpopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguistics (ACL-IJCNLP)*, pp. 993–1003, 2021.

[9] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "Hubert: Self-supervised speech representation learning by masked prediction of hidden units," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, p. 3451–3460, Oct. 2021.

[10] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 28492–28518, 2023. [Online]. Available: https://proceedings.mlr.press/v202/radford23a.html.

[11] J. Fiscus, "A post-processing system to yield reduced word error rates: Recognizer output voting error reduction (rover)," in *1997 IEEE Workshop on Automatic Speech Recognition and Understanding Proceedings*, pp. 347–354, 1997.